PENGOLAHAN CITRA DIGITAL (3 SKS / 150 MENIT)

Minggu ke-6: Sharpening dan Edge Enhancement pada Citra Digital

Sub-CPMK 6: Menerapkan operator turunan pertama Sobel, turunan kedua Laplacian, Unsharp Masking / High-Boost, dan Canny Edge Detector dengan Python.

Endah Kristiani, Ph.D.

Struktur Perkuliahan & Alokasi Waktu (150 Menit)

Sesi 1: Teori Turunan Pertama, Kedua & Algoritma Canny (50 Menit)

Aktivitas: Gradien Sobel/Prewitt (|G|, theta), Laplacian 2D, Unsharp Masking & High-Boost, serta 5 Tahapan Canny Edge Detector.

Sesi 2: Studi Kasus Nyata & Bedah Kode Python (50 Menit)

Aktivitas: Studi kasus retak tulang medis, pemetaan satelit GIS, live coding cv2.Sobel, cv2.Laplacian, cv2.Canny, dan High-Boost.

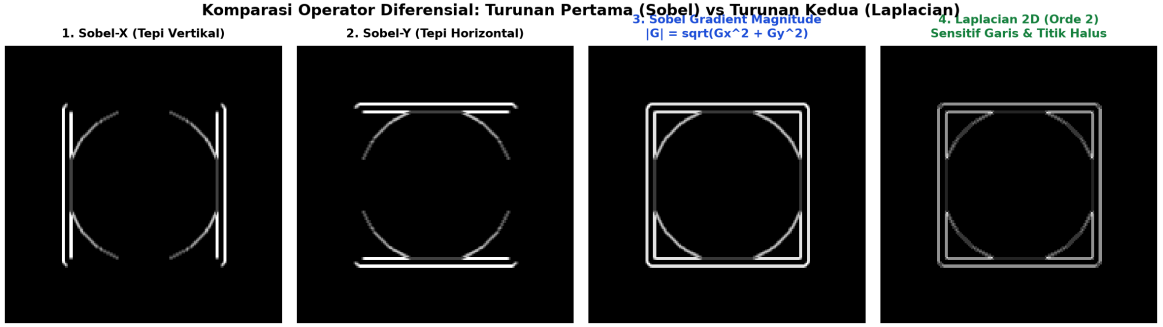
Sesi 3: Praktikum Mandiri Mahasiswa & Analisis Tepi (50 Menit)

Aktivitas: Penajaman rontgen tulang X-Ray (High-Boost) dan ekstraksi kontur batas jalan citra satelit (Canny).

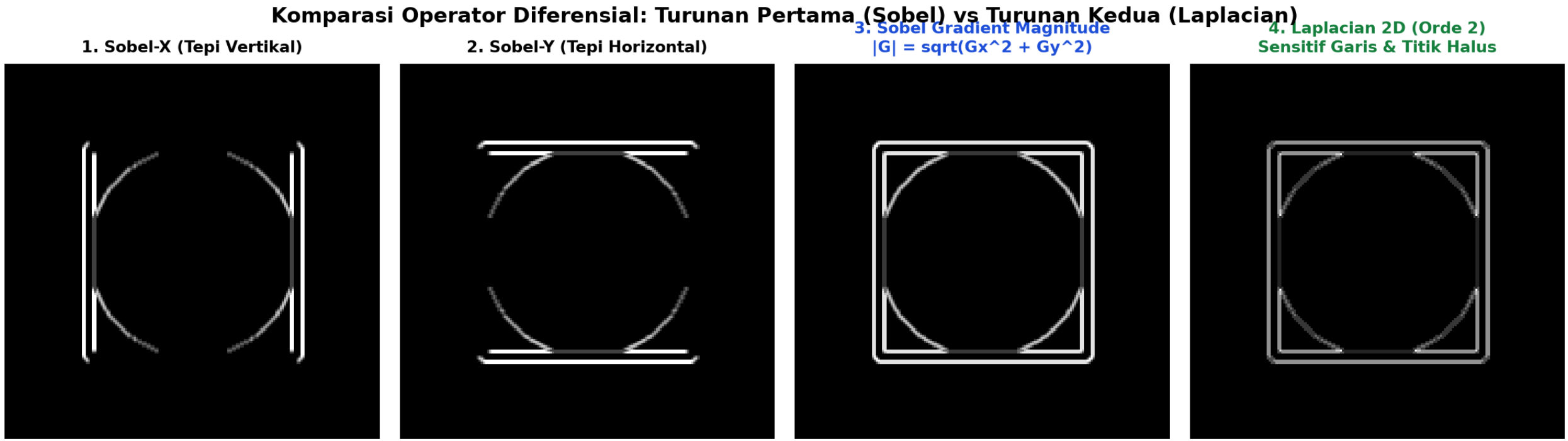
Operator Diferensial: Turunan Pertama vs Turunan Kedua

Perbedaan Sobel vs Laplacian

- 1. Turunan Pertama (Gradien Sobel):
  - Mendeteksi keberadaan tepi dengan menghitung magnitudo |G| = sqrt(Gx^2 + Gy^2) dan arah sudut theta = arctan(Gy/Gx).
  - Menghasilkan garis tepi yang tebal dan tahan derau.
- 2. Turunan Kedua (Laplacian 2D):
  - Mendeteksi transisi perubahan (zero crossing).
  - Sangat sensitif terhadap detail garis halus, sudut lancip, dan titik diskontinuitas.
  - Rekonstruksi Sharpening: g(x, y) = f(x, y) - del^2 f(x, y).



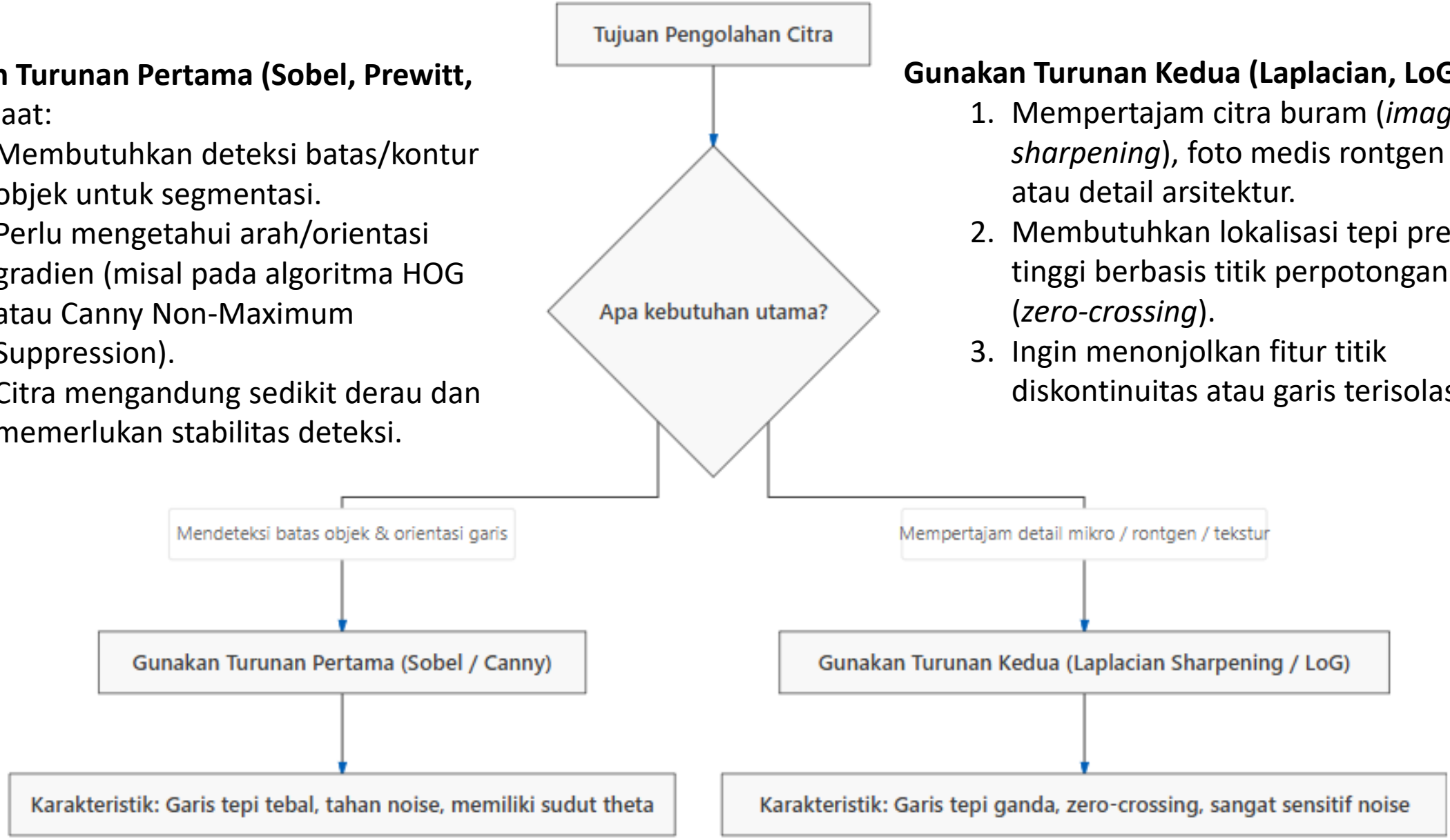
Operator Diferensial: Turunan Pertama vs Turunan Kedua



Kapan Menggunakan Masing-Masing Operator?

Gunakan Turunan Pertama (Sobel, Prewitt, Canny) saat:

1. Membutuhkan deteksi batas/kontur objek untuk segmentasi.
2. Perlu mengetahui arah/orientasi gradien (misal pada algoritma HOG atau Canny Non-Maximum Suppression).
3. Citra mengandung sedikit derau dan memerlukan stabilitas deteksi.



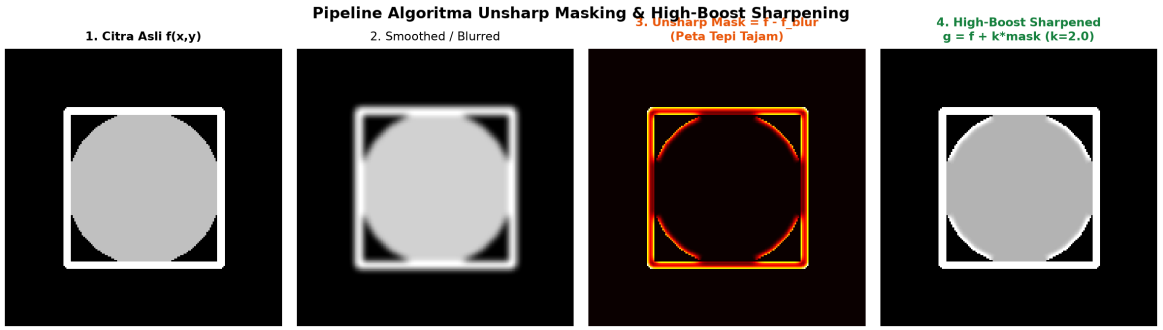
Gunakan Turunan Kedua (Laplacian, LoG) saat:

1. Mempertajam citra buram (*image sharpening*), foto medis rontgen X-ray, atau detail arsitektur.
2. Membutuhkan lokalisasi tepi presisi tinggi berbasis titik perpotongan nol (*zero-crossing*).
3. Ingin menonjolkan fitur titik diskontinuitas atau garis terisolasi.

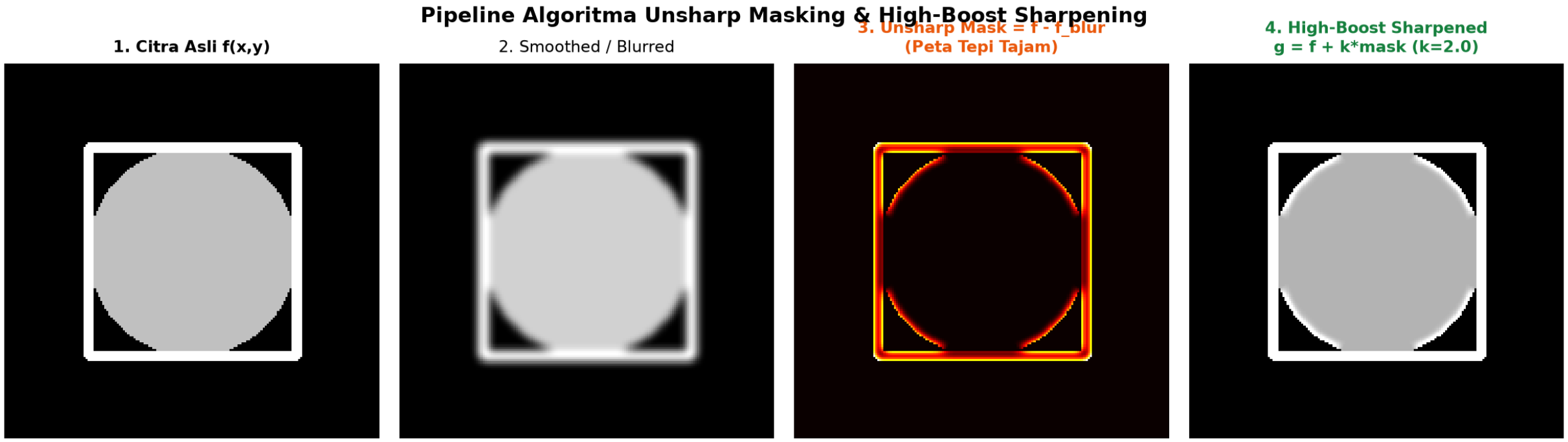
Unsharp Masking & High-Boost Filtering

Tahapan Alur Unsharp Masking

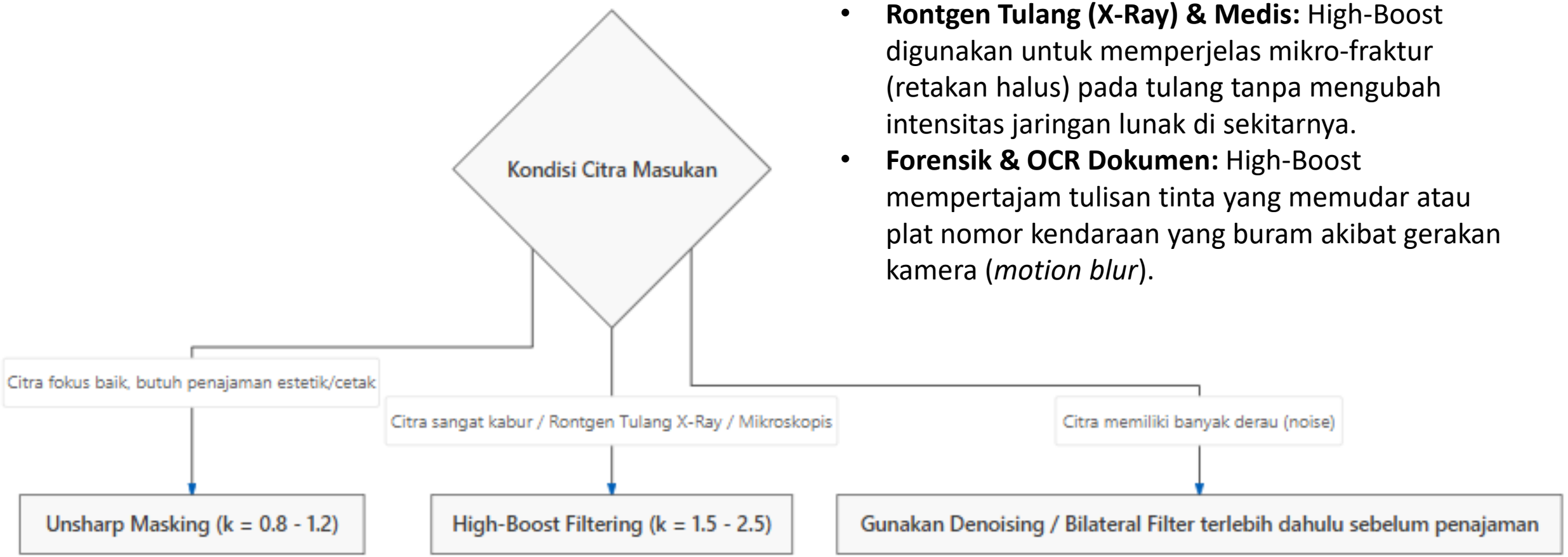
- 1. Tahap 1: Lakukan penghalusan citra asli -> f_blur(x, y).
- 2. Tahap 2: Kurangkan citra asli dengan citra blur untuk menghasilkan Mask Tepi:
  mask(x, y) = f(x, y) - f_blur(x, y)
- 3. Tahap 3: Tambahkan mask tepi kembali ke citra asli dengan faktor penguat k:
  g(x, y) = f(x, y) + k . mask(x, y)
- Kategori Nilai k:
- k = 1 : Unsharp Masking Standar
- k > 1 (misal k=2.0) : High-Boost Filtering (Sangat Tajam)



Unsharp Masking & High-Boost Filtering



Skenario Penggunaan



- **Rontgen Tulang (X-Ray) & Medis:** High-Boost digunakan untuk memperjelas mikro-fraktur (retakan halus) pada tulang tanpa mengubah intensitas jaringan lunak di sekitarnya.
- **Forensik & OCR Dokumen:** High-Boost mempertajam tulisan tinta yang memudar atau plat nomor kendaraan yang buram akibat gerakan kamera (*motion blur*).

Canny Edge Detector: Gold Standard Deteksi Tepi Presisi

5 Tahapan Algoritma Canny

- 1. Gaussian Filter: Menghilangkan derau frekuensi tinggi.
- 2. Gradient Calculation: Menghitung Sobel |G| dan arah sudut theta.
- 3. Non-Maximum Suppression (NMS): Menipiskan tepi tebal menjadi garis selebar 1 piksel (membuang piksel lokal non-maksimum).
- 4. Double Thresholding: Menetapkan ambang T_high (tepi kuat) dan T_low (tepi lemah).
- 5. Edge Tracking by Hysteresis: Mempertahankan tepi lemah hanya jika terhubung secara spasial dengan tepi kuat.





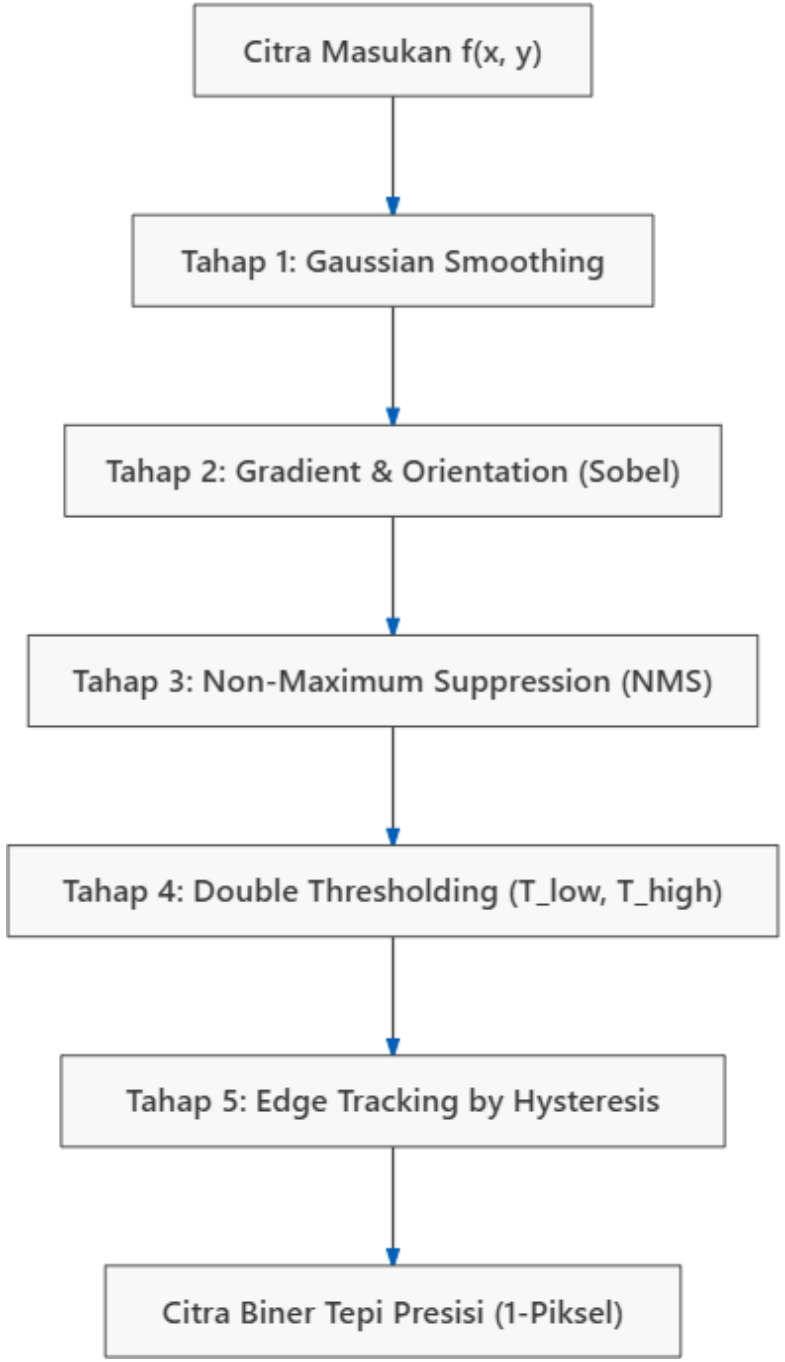
Canny Edge Detector: Gold Standard Deteksi Tepi Presisi

Tiga Kriteria Desain Optimal John F. Canny
John Canny merumuskan 3 kriteria matematis yang harus dipenuhi oleh sebuah detektor tepi yang ideal:

1. **Tingkat Kesalahan Rendah (*Low Error Rate / Good Detection*):**
   Maksimal dalam mendeteksi tepi asli (*true edges*) dan minimal dalam mendeteksi tepi palsu akibat derau (*false positives*) atau melewatkan tepi penting (*false negatives*).
2. **Lokalisasi Presisi (*Good Localization*):**
   Jarak antara posisi tepi yang terdeteksi oleh algoritma dengan pusat tepi fisik sebenarnya pada objek harus seminimal mungkin.
3. **Respon Tunggal (*Single Edge Response*):**
   Satu garis tepi pada objek hanya boleh menghasilkan **satu garis tepi selebar 1 piksel** (tidak boleh menghasilkan tepi yang melebar tebal atau ganda).

Diagram Alur 5 Tahapan Algoritma Canny

- Tahap 1: Gaussian Smoothing (Peredaman Derau)
  Operasi turunan/diferensiasi sangat sensitif terhadap derau frekuensi tinggi. Sebelum dihitung gradiennya, citra harus dihaluskan terlebih dahulu menggunakan Gaussian Filter 2D.
- Tahap 2: Gradient Magnitude & Orientation (Kalkulasi Gradien)
  Menghitung laju perubahan intensitas di sumbu horizontal (Gx) dan vertikal (Gy) menggunakan kernel Sobel.
- Tahap 3: Non-Maximum Suppression (NMS — Penipisan Tepi)
  Hasil dari Tahap 2 menghasilkan tepi yang tebal dan kabur. NMS bertugas menipiskan lereng gradien menjadi garis setajam 1 piksel.
- Tahap 4: Double Thresholding (Klasifikasi Ambang Ganda)
  Untuk menyaring sisa derau dan variasi pencahayaan, Canny menggunakan dua nilai ambang batas: ambang rendah dan tinggi, biasanya dengan rasio 1:2 atau 1:3
- Tahap 5: Edge Tracking by Hysteresis (Pelacakan Kontur Histeresis)
  Tahap ini menyelesaikan ambiguitas pada kelompok Weak Edge:
  Algoritma melakukan penelusuran konektivitas 8-tetangga (8-connected neighborhood):
  - Jika sebuah Weak Edge terhubung secara fisik dengan Strong Edge (baik langsung maupun berantai), maka piksel tersebut dipromosikan menjadi tepi sejati.
  - Jika sebuah Weak Edge berdiri sendiri (terisolasi) tanpa koneksi ke Strong Edge manapun, piksel tersebut dihapus/dieliminasi karena merupakan artefak derau.



Implementasi & Tips Tuning Parameter di Python (OpenCV)

```python
import cv2
import numpy as np

# 1. Baca citra grayscale
img = cv2.imread('road_satellite.jpg', cv2.IMREAD_GRAYSCALE)

# 2. Canny Edge Detection Standar
# threshold1 = T_low (50), threshold2 = T_high (150)
edges = cv2.Canny(img, threshold1=50, threshold2=150, apertureSize=3, L2gradient=True)

# 3. Auto-Canny (Penentuan Threshold Otomatis berbasis Median Citra)
def auto_canny(image, sigma=0.33):
    v = np.median(image)
    lower = int(max(0, (1.0 - sigma) * v))
    upper = int(min(255, (1.0 + sigma) * v))
    return cv2.Canny(image, lower, upper, L2gradient=True)

auto_edges = auto_canny(img)
```

1. **Jika tepi terlalu banyak bintik derau:** Naikkan T_{low} atau perbesar ukuran kernel Gaussian pra-pemrosesan (misal cv2.GaussianBlur(img, (7,7), 2.0)).
2. **Jika garis kontur objek terputus-putus:** Turunkan T_{low} untuk memudahkan pelacakan *hysteresis*, atau turunkan T_{high} jika objek memiliki kontras yang redup.
3. **Gunakan L2gradient=True:** OpenCV secara default menggunakan hampiran $L_1 = |G_x| + |G_y|$; mengaktifkan L2gradient=True akan menggunakan formula Euclidean murni $\sqrt{G_x^2 + G_y^2}$ yang lebih akurat secara geometri.

Studi Kasus 6A (Medis): Deteksi Retak Tulang dengan High-Boost

Penajaman Retak Tulang Halus

- Masalah (Before): Rontgen sinar-X tulang sedikit buram. Garis retak rambut halus (hairline bone fracture) memiliki kontras transisi tipis yang mudah terlewat oleh dokter.
- Solusi (High-Boost Filtering k = 2.0):
  - Mengekstrak peta mask tepi frekuensi tinggi: mask = f - blur.
  - Menambahkan mask dengan penguatan k=2.0: g = f + 2.0 * mask.
- Hasil (After): Garis retak tulang menyala tajam dan kontras, meminimalkan risiko salah diagnosis pada pasien darurat.



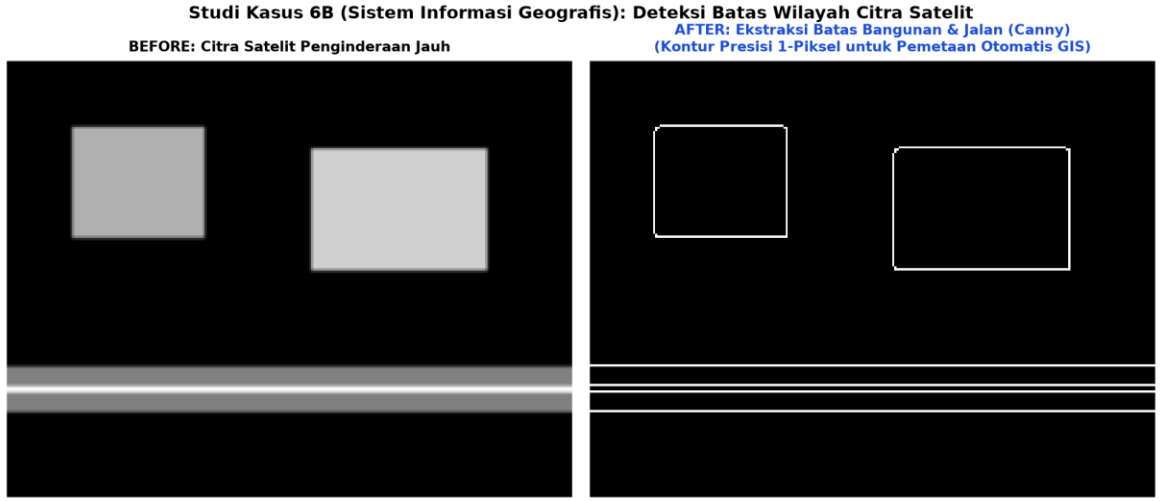
Studi Kasus 6B (SIG): Deteksi Batas Jalan Citra Satelit (Canny)

Pemetaan Wilayah GIS Otomatis

• Masalah (Before): Citra satelit resolusi tinggi memiliki jutaan piksel bertekstur rumit (atap rumah, pepohonan, aspal).

• Solusi (Canny Edge Detector 5-Tahap):
  • Non-Maximum Suppression (NMS) menipiskan tepi menjadi 1 piksel presisi.
  • Hysteresis Thresholding menghubungkan jalan panjang tanpa bintik palsu.

• Hasil (After): Vektorisasi batas jalan dan blok bangunan berhasil diekstrak otomatis untuk pembaruan peta navigasi Google Maps / OpenStreetMap.



Kode Python Sharpening & Edge Detection

Fungsi Deteksi & Sharpening

- 1. Sobel Magnitudo: cv2.Sobel(..., cv2.CV_64F)
- 2. Laplacian: cv2.Laplacian(..., cv2.CV_64F)
- 3. Unsharp Mask: cv2.addWeighted(img, 1.5, blur, -0.5, 0)
- 4. Canny Edges: cv2.Canny(gray, threshold1=50, threshold2=150)

Python: edge_and_sharpening_demo.py

```
import cv2, numpy as np, matplotlib.pyplot as plt

img = cv2.imread('xray_bone.jpg', cv2.IMREAD_GRAYSCALE)

# 1. Sobel Edge Magnitude
sx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
sy = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)
sobel_mag = np.uint8(np.clip(np.hypot(sx, sy), 0, 255))

# 2. Laplacian Sharpening
lap = cv2.Laplacian(img, cv2.CV_64F, ksize=3)
lap_sharp = np.uint8(np.clip(img.astype(np.float64) - lap, 0, 255))

# 3. Unsharp Masking / High-Boost (k=1.5)
blur = cv2.GaussianBlur(img, (5, 5), 1.5)
unsharp_out = cv2.addWeighted(img, 2.5, blur, -1.5, 0)

# 4. Canny Edge Detection
canny_out = cv2.Canny(img, 50, 150)

# Visualisasi
fig, axs = plt.subplots(1, 4, figsize=(15, 4))
axs[0].imshow(img, cmap='gray'); axs[0].set_title('Original')
axs[1].imshow(sobel_mag, cmap='gray'); axs[1].set_title('Sobel Magnitude')
axs[2].imshow(unsharp_out, cmap='gray'); axs[2].set_title('High-Boost (k=1.5)')
axs[3].imshow(canny_out, cmap='gray'); axs[3].set_title('Canny (1-px Tepi)')
for ax in axs: ax.axis('off')
plt.tight_layout(); plt.show()
```